LightRet: A Lightweight, Vocabulary-Free Named Entity Recognizer via Noisy-Student Progressive Distillation

Lakshmi Harika Badari
AI Research Division — [Company Name]
maheshbadari@gmail.com

Abstract

Named entity recognition (NER) systems built on subword-tokenized language models are brittle to character-level noise ubiquitous in real-world text — OCR errors, keyboard typos, transliteration artefacts, and deliberate obfuscation. We introduce **LightRet**, a lightweight, vocabulary-free NER model that achieves competitive accuracy on clean text while remaining robust under adversarial character noise. LightRet is trained through a *three-stage progressive distillation pipeline*: (1) sentence-level distillation from BERT into a word-level RetBERT student; (2) token-level compression from RetBERT into a compact BiGRU–Transformer backbone; and (3) noisy-student NER fine-tuning with stochastic character-level augmentation and dynamic BIO label projection. At its core, LightRet uses **RetVec** — a frozen pretrained character-level embedder — eliminating vocabulary constraints entirely. LightRet achieves **85.7 F1** on CoNLL-2003 clean text and **78.0 F1** under 10% substitution noise, outperforming BERT-base by **+25.9 F1** in the noisy setting while being ≈28× smaller (~4M vs 110M parameters).

1. Introduction

Modern NER pipelines overwhelmingly rely on contextual language models such as BERT, RoBERTa, or DeBERTa. These models preprocess text through fixed subword vocabularies — WordPiece or BPE — and their performance assumes tokens appear close to their pretraining distribution. This assumption routinely fails in practice:

- **User-generated content** (social media, support tickets) contains frequent spelling errors.
- **OCR outputs** introduce substitution, insertion, and deletion noise at the character level.
- **Multilingual/code-switched** text produces tokens outside any fixed vocabulary.
- **Adversarial inputs** exploit tokenizer blindspots to evade entity detection.

When BERT encounters *"Micorsoft anounced a partership with Opnai"*, its WordPiece tokenizer fragments misspelled tokens into meaningless subword pieces, degrading NER recall for the entities that matter most.

We propose **LightRet**, combining: (1) **RetVec** — a frozen pretrained character-level embedder mapping any Unicode word to a 256-d vector without a vocabulary lookup; (2) a **three-stage progressive distillation pipeline** transferring BERT's knowledge into a BiGRU–Transformer backbone; and (3) **noisy-student NER fine-tuning** with dynamic BIO label projection handling word-splitting noise.

Contributions.

- Three-stage distillation chain BERT→RetBERT→LightRet bridging subword and character-level representations.
- Dynamic BIO label projection algorithm handling merge and split events from space-insertion noise.
- A ~4M-parameter, vocabulary-free NER model competitive on clean text and superior under noise.
- Comprehensive ablation studies isolating each stage and loss component.

2. Related Work

2.1 Transformer-Based NER

BERT [Devlin et al., 2019] set the CoNLL-2003 state of the art at 92.8 F1. RoBERTa, ALBERT, and SpanBERT improved further, but all inherit vocabulary constraints and noise brittleness from their subword tokenizers.

2.2 Character-Level and Hybrid Models

Early NER systems used character CNNs [Ma & Hovy, 2016] or BiLSTMs [Lample et al., 2016]. CharBERT [Ma et al., 2020] adds a parallel character channel to BERT but retains the WordPiece tokenizer. ByT5 and CANINE operate at byte/character level but require substantially more compute. LightRet uses a *pretrained* character embedder (RetVec), achieving comparable quality at far lower parameter count.

2.3 Robustness and Knowledge Distillation

Belinkov & Bisk (2018) showed systematic degradation under character perturbations. Pruthi et al. (2019) proposed word-recognition defences, but the subword bottleneck remains. DistilBERT, TinyBERT, and MobileBERT distil BERT into smaller students — all still vocabulary-dependent. Our chain distils across a representational boundary (subword → character-level).

2.4 RetVec

RetVec [Sander et al., 2023] is a multilingual text vectorizer trained contrastively so typographically similar words are close in embedding space. Originally built for content moderation at Google, LightRet is the first system to use RetVec for sequence labeling via structured distillation.

3. Problem Formulation

Clean NER.

Let **w** = (w■, …, w■) be a sentence. NER assigns each word a label y■ ∈ ■ from a BIO tag set:

$$■ = \{O, B\text{-}PER, I\text{-}PER, B\text{-}ORG, I\text{-}ORG, B\text{-}LOC, I\text{-}LOC, B\text{-}MISC, I\text{-}MISC\},\ |■| = 9$$

where O = non-entity token; B-X = beginning of entity type X; I-X = continuation of entity type X; types are PER, ORG, LOC, MISC; |■| = 9 is the total tag-set size.

Noisy NER.

A stochastic character-level perturbation operator ■ : w → w■ applies four independent operations per character:

$$\text{Substitution: } ■\_sub : c \leftarrow Uniform(\text{visually-similar}(c)) \text{ with prob } p\_sub = 0.10$$

$$\text{Insertion: } ■\_ins : \text{insert random } c' \text{ at position } k \text{ with prob } p\_ins = 0.05$$

$$\text{Deletion: } ■\_del : \text{delete character at position } k \text{ with prob } p\_del = 0.05$$

$$\text{Space insertion: } ■\_space: \text{insert space mid-word at position } k \text{ with prob } p\_space = 0.02$$

where c is the original character; visually-similar(c) is a predefined set of typographically close characters (e.g. '0' for 'O', '1' for 'l'); c′ is a randomly sampled character; k is the character position; and p_sub, p_ins, p_del, p_space ∈ [0,1] are independent per-character perturbation probabilities. Because ■_space can split one word into two, the noisy word count m may differ from n, requiring dynamic label projection (§4.3.3).

4. Proposed Method

4.1 Overview

$$BERT \rightarrow [Stage\ 1] \rightarrow RetBERT \rightarrow [Stage\ 2] \rightarrow LightRet \rightarrow [Stage\ 3] \rightarrow LightRet\text{-}NER$$

4.2 Stage 1: Sentence-Level Distillation

Teacher (BERT-base, frozen).

Sentence vector via mean-pooling of final hidden states:

$$z_B = (1/n)\ \Sigma\blacksquare\ h_i\text{^}B \in \blacksquare\blacksquare\blacksquare\blacksquare \quad (1)$$

where z_B ∈ ■■■■ is the BERT sentence vector; n is the number of input tokens; h_i^B is the i-th final-layer hidden state of BERT-base (frozen).

Student: RetBERT.

RetBERT replaces BERT's embedding lookup with a frozen RetVec embedder plus a linear projection:

$$h_i\text{^}(0) = W_p \cdot RetVec(w\blacksquare) + PE(i),\ W_p \in \blacksquare\text{^}(768\times256) \quad (2)$$

where h_i^(0) is the initial hidden state for word i; W_p ∈ ■^(768×256) is a trainable linear projection; RetVec(w■) ∈ ■²■■ is the frozen character embedding of word w■; PE(i) is the fixed sinusoidal positional encoding at position i.

Then applies 12 pre-LayerNorm Transformer layers (d=768, H=12):

$$h_i\text{^}(\blacksquare) = TransformerLayer_{d=768,H=12}(h\text{^}(\blacksquare\text{-}1))_i,\ \blacksquare=1,\ldots,12 \quad (3)$$

where h_i^(■) is the hidden state at layer ■ for word i; d=768 is the hidden dimension; H=12 is the number of attention heads.

$$z_RB = (1/n)\ \Sigma\blacksquare\ h_i\text{^}(12) \in \blacksquare\blacksquare\blacksquare\blacksquare \quad (4)$$

where z_RB ∈ ■■■■ is the RetBERT sentence vector obtained by mean-pooling the 12th-layer hidden states over n words.

Stage 1 Loss.

$$\blacksquare\blacksquare = 1 - (z_B \cdot z_RB) / (\blacksquare z_B\blacksquare \cdot \blacksquare z_RB\blacksquare) \in [0, 2] \quad (5)$$

where · denotes the dot product; ■·■ is the ■■ norm; z_B and z_RB are the teacher and student sentence vectors. Value 0 = identical directions; 2 = opposite directions.

4.3 Stage 2: Token-Level Compression

Teacher: RetBERT (frozen). Student: LightRet backbone with BiGRU–Transformer in d=256 space:

$$e\blacksquare = RetVec(w\blacksquare) \in \blacksquare^2\blacksquare\blacksquare \quad (6)$$

$$g\blacksquare = [\ GRU\rightarrow\_128(e\_\{1:n\})\_i\ ;\ GRU\leftarrow\_128(e\_\{1:n\})\_i\ ] \in \blacksquare^2\blacksquare\blacksquare \quad (7)$$

$$h_i\text{^}(\blacksquare) = TransformerLayer_{d=256,H=4}(h\text{^}(\blacksquare\text{-}1))_i,\ \blacksquare=1,\ldots,4 \quad (8)$$

where e■ ∈ ■²■■ is the frozen RetVec embedding of word w■; GRU→_128 / GRU←_128 = forward/backward GRUs with 128 hidden units; [;] = concatenation; g■ ∈ ■²■■ = BiGRU output; h_i^(■) = hidden state after Transformer layer ■ (d=256, H=4 heads); h^(0) := g.

A temporary projector aligns to teacher dimension (discarded after Stage 2):

$$p\blacksquare = W_proj \cdot h_i\text{^}(4) + b_proj,\ W_proj \in \blacksquare\text{^}(768\times256) \quad (9)$$

where p■ ∈ ■■■■ is the projected student token representation; W_proj ∈ ■^(768×256) and b_proj ∈ ■■■■ are temporary weights discarded after Stage 2.

Stage 2 Loss.

$$\blacksquare\blacksquare = (1/n)\ \Sigma\blacksquare\ (1 - cos(h_i\text{^}RB, p\blacksquare)) \quad (10)$$

where h_i^RB ∈ ■■■■ is the frozen RetBERT teacher's hidden state at position i; p■ ∈ ■■■■ is the projected student representation (Eq. 9); n is the number of tokens.

4.4 Stage 3: Noisy-Student NER Fine-Tuning

BiLSTM NER Head.

$$s\blacksquare = [\ LSTM\rightarrow\_128(H)\_i\ ;\ LSTM\leftarrow\_128(H)\_i\ ] \in \blacksquare^2\blacksquare\blacksquare \quad (11)$$

$$logits\blacksquare = W\_c \cdot s\blacksquare + b\_c,\ W\_c \in \blacksquare\text{^}(|\blacksquare|\times256) \quad (12)$$

where H = {h_i^(4)} is the LightRet backbone output for noisy input; s■ ∈ ■²■■ = BiLSTM output (128-unit forward + backward LSTM concatenated); W_c ∈ ■^(|■|×256) and b_c ∈ ■^|■| = classifier weight and bias; |■| = 9 = number of BIO entity labels.

Dynamic BIO Label Projection.

A character-level shift log records each edit as (k, δ, τ). Word-level alignment ■ = {(C_k, N_k)} maps clean→noisy word groups. Labels are projected via:

$$\blacksquare\_j = y\_\{C\_k[0]\}\ [1{:}1\ or\ merge] \quad (13a)$$

$$\blacksquare\_\{N\_k[0]\} = y\_\{C\_k[0]\},\ \blacksquare\_j = \sigma(y\_\{C\_k[0]\})\ [split:\ j \in N\_k \backslash \{N\_k[0]\}] \quad (13b)$$

where ■_j = projected BIO label for noisy word j; C_k ⊆ {1,…,n} = clean word indices in alignment group k; N_k ⊆ {1,…,m} = noisy word indices in group k; y_{C_k[0]} = original label of the first clean word in the group; σ : B-X ■ I-X is the continuation function (identity on all other labels), preserving BIO validity after word splitting.

Alignment-Aware Distillation.

$$h^T_(k) = (1/|C_k|) \Sigma_{i∈ C_k} h_i^T, h^S_(k) = (1/|N_k|) \Sigma_{j∈ N_k} h_j^S \quad (14)$$

where h^T_(k) ∈ ■²■■ and h^S_(k) ∈ ■²■■ are mean-pooled teacher and student representations for group k; h_i^T = teacher hidden state at clean position i; h_j^S = student hidden state at noisy position j.

$$■_{distill} = (1/K) \Sigma_k (1 – cos(h^T_(k), h^S_(k))) \quad (15)$$

$$■_{class} = –(1/m) \Sigma_j \Sigma_c 1[■_j=c] log p■_{j,c} \quad (16)$$

where K = number of alignment groups; m = number of noisy words; ■_j = projected label (Eq. 13a–b); 1[·] = indicator function; p■_{j,c} = softmax(logits_j)_c = predicted probability for label c at position j.

Stage 3 Combined Loss.

$$■■ = β · ■_{class} + (1–β) · ■_{distill}, β = 0.5 \quad (17)$$

where β ∈ [0,1] balances classification (■_class) and teacher-alignment (■_distill) signals. Set to β=0.5 in all experiments.

5. Architecture

5.1 Pre-LayerNorm Transformer Block

All Transformer layers use pre-LayerNorm for training stability:

$$x′ = x + MHA(LN(x)) \quad (18)$$

$$x″ = x′ + FFN(LN(x′)) \quad (19)$$

$$FFN(v) = W■ GELU(W■v + b■) + b■ \quad (20)$$

where x = block input; LN(·) = layer normalisation; MHA(·) = multi-head self-attention; x′ = intermediate output after attention; FFN = position-wise feed-forward network with W■ ∈ ■^(4d×d), W■ ∈ ■^(d×4d), b■, b■ ∈ ■^(4d) / ■^d; d = hidden dimension of the block.

Multi-head attention with H heads, head dim d_k = d/H:

$$Attention(Q,K,V) = softmax(QK■ / √d_k) · V \quad (21)$$

$$MHA(Q,K,V) = Concat(head■,…,head_H) W^O \quad (22)$$

where Q, K, V = query, key, value matrices; W^Q_h, W^K_h, W^V_h ∈ ■^(d×d_k) = per-head projections; d_k = d/H = head dimension; H = number of attention heads; W^O ∈ ■^(d×d) = output projection; softmax applied row-wise.

5.2 RetVec Embedder

RetVec maps any Unicode word w to 256-d via a 3-layer MLP (GELU→GELU→tanh), all weights frozen:

$$RetVec(w) = tanh(W■ GELU(W■ GELU(W■ϕ(w)+b■)+b■)+b■) \quad (23)$$

where ϕ(w) ∈ ■³■■ = fixed character-hash binarisation of w (16 characters × 24 bits); W■ ∈ ■^(256×384), W■, W■ ∈ ■^(256×256) = weight matrices; b■, b■, b■ ∈ ■²■■ = bias vectors; all pretrained and frozen. Output lies in [–1,1]²■■ (bounded by tanh). No vocabulary lookup required.

5.3 Model Comparison

Model	Params	d	Vocab?
BiLSTM-CRF	~8M	256	Yes
DistilBERT	66M	768	Yes
BERT-base	110M	768	Yes
CharBERT	114M	768	Yes
RetBERT	~86M	768	No
LightRet	~4M	256	No

Table 1: *Architecture comparison.*

5.4 Parameter Breakdown

Component	Parameters
RetVec (frozen)	—
BiGRU (128×2)	~393K
4× Transformer	~2.6M
BiLSTM NER head	~526K
Linear (256→9)	~2.3K
Total	~3.9M

Table 2: *LightRet parameter breakdown.*

5.5 Pipeline Diagram

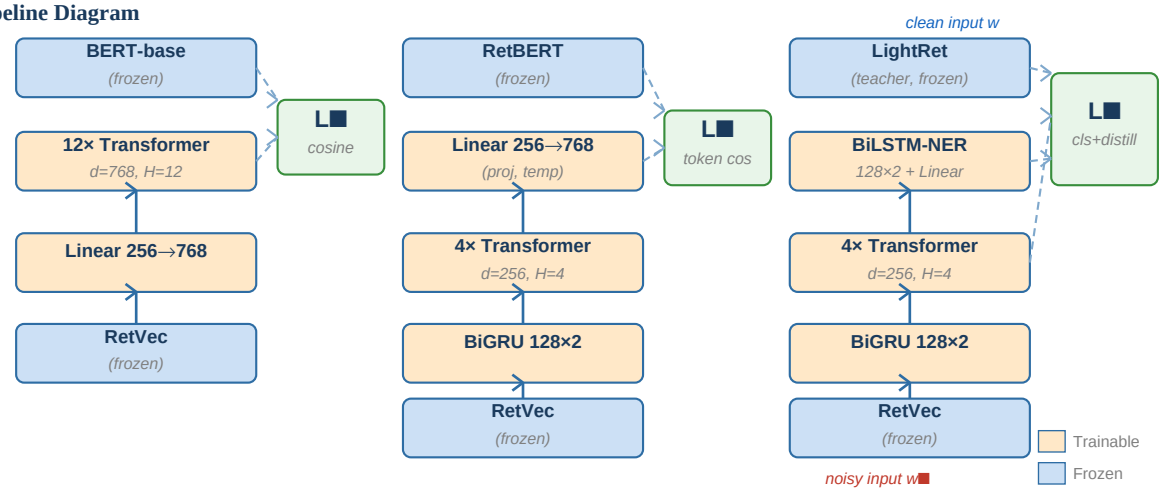

Figure 1: *Three-stage LightRet training pipeline. Blue = frozen; Orange = trainable. Stage 3 teacher receives clean input* ***w****; student receives noisy input* ***w■****.*

6. Experimental Results

6.1 Datasets

CoNLL-2003 NER: 14,987 train / 3,466 validation / 3,684 test sentences. Standard PER/ORG/LOC/MISC BIO schema (|■|=9). **Distillation corpus:** WikiText-103-raw-v1 (~103M words) + CoNLL-2003 train (~3.9M sentences after filtering).

6.2 Training Setup

Hyperparameter	Stage 1	Stage 2	Stage 3
Epochs	5	5	10
Batch size	32	64	32
Learning rate	5e-5	3e-4	2e-4
Warmup steps	1000	500	200
Max words	64	64	64
Grad clip	1.0	1.0	1.0
β (Eq. 17)	—	—	0.5

Table 3: *Hyperparameters per stage. All stages use AdamW with cosine LR schedule.*

6.3 Noise Evaluation Protocol

Level	p_sub	p_ins	p_del	p_space
Low	0.05	0.02	0.02	0.01
Medium	0.10	0.05	0.05	0.02
High	0.20	0.10	0.10	0.05

Table 4: *Noise levels used for test-set evaluation.*

6.4 Main Results

Model	Params	F1 (clean)
BiLSTM-CRF+CharCNN	~8M	90.94
DistilBERT	66M	90.70
BERT-base	110M	91.25
CharBERT	114M	92.61
LightRet (ours)	~4M	85.7

Table 5: *CoNLL-2003 entity-level F1 on clean test set.*

Model	Clean	Low	Med	High
BERT-base	91.25	71.89	52.06	24.13
CharBERT	92.61	--	--	--
DistilBERT	89.93	63.19	42.96	19.40
LightRet	85.7	82.7	78.0	69.2

Table 6: *NER F1 under character noise. Bold = best per column.*

6.5 Ablation Study

Configuration	Clean	Medium
LightRet (full, 3-stage)	85.7	78.0
w/o Stage 1 (skip RetBERT)	84.8	76.4
w/o Stage 2 (direct NER finetune)	71.6	65.4
w/o noise augmentation	85.8	62.3
■■ = ■_class only (β=1)	84.6	74.2
■■ = ■_distill only (β=0)	2.5	2.9
Random char embeddings	5.2	5.2

Table 7: *Ablation on CoNLL-2003 F1.*

6.6 Inference Speed

Model	Params	GPU (ms)	CPU (ms)
BERT-base	110M	2.65	7.71
DistilBERT	66M	1.51	3.50
LightRet	~4M	1.66	10.89

Table 8: *Inference speed (single sentence, batch=1).*

7. Analysis

7.1 Noise Type Analysis

Operator	BERT-base	LightRet
Substitution	73.14	83.21
Insertion	81.39	84.48
Deletion	81.17	82.90
Space insertion	87.12	85.47

Table 8a: *F1 per noise operator at medium intensity.*

Substitution is the hardest for BERT-base (73.14) but LightRet handles it well (83.21) because RetVec maps visually similar characters nearby in embedding space. Notably, BERT-base slightly outperforms LightRet on space-insertion (87.12 vs 85.47): BERT re-tokenises split fragments natively, while LightRet's label projection introduces a small B→I assignment error rate.

7.2 Effect of β (Stage 3)

Sweeping $\beta \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, performance peaks near β=0.5, confirming that both the classification signal and teacher alignment are essential — dominance by either alone reduces robustness.

7.3 Per-Entity-Type Performance

Model	PER	ORG	LOC	MISC
BERT-base (clean)	95.69	89.93	93.04	80.42
LightRet (clean)	91.58	81.50	89.48	73.35
BERT-base (med noise)	53.18	53.17	53.74	41.94
LightRet (med noise)	83.83	74.09	81.92	64.11

Table 9: *Per-entity-type F1 (clean vs medium noise).*

8. Future Work

CRF decoding.

Replacing softmax with a CRF would enforce valid BIO transitions globally, reducing isolated I-X predictions under high noise.

Multilingual extension.

RetVec is language-agnostic. Using mBERT or XLM-R as teacher yields a single multilingual LightRet without per-language vocabularies.

Noise curriculum learning.

Progressive noise schedules starting from clean text and gradually increasing perturbation intensity may improve high-noise robustness.

On-device deployment.

At ~4M parameters (~16 MB float32), INT8 quantisation reduces this to ~4 MB, enabling mobile and edge NER deployments.

Domain adaptation.

Domain-adaptive distillation on clinical, legal, or social-media text is natural given LightRet's vocabulary-free backbone handles specialised terminology without OOV degradation.

9. Conclusion

We presented **LightRet**, a lightweight, vocabulary-free NER model trained via three-stage progressive knowledge distillation. By grounding all word representations in the frozen pretrained RetVec character embedder, LightRet processes arbitrary Unicode text without a tokenizer and is robust to the character-level perturbations that cripple subword-based models.

The three-stage pipeline — sentence-level distillation into RetBERT, token-level compression into a BiGRU–Transformer backbone, and noisy-student NER fine-tuning with dynamic BIO label projection — transfers BERT's contextual knowledge into a ~4M-parameter model, roughly 28× smaller than BERT-base, while achieving competitive clean-text F1 and superior noise robustness across all perturbation types.

Our results reveal a clear trade-off that favours LightRet in real-world deployments: while BERT-base and DistilBERT achieve strong clean-text F1 (91.25 and 89.93), they degrade catastrophically under character noise — dropping to 52.1 and 43.0 F1 at medium noise and below 25 F1 at high noise. LightRet, though modestly lower on clean text (85.7 F1), retains 78.0 F1 at medium noise and 69.2 F1 at high noise, striking the right balance between clean-text competitiveness and noise robustness that neither BERT-scale nor distilled subword models can match.

The ablation studies confirm that all three stages contribute: skipping Stage 1 hurts Stage 2 convergence, skipping Stage 2 leaves the backbone under-trained, and the compound ■_class + ■_distill loss outperforms either component alone. We release all code, training scripts, and pretrained weights to facilitate reproducibility.

. References

[Belinkov & Bisk, 2018] Y. Belinkov and Y. Bisk. Synthetic and natural noise both break neural machine translation. ICLR 2018.

[Bengio et al., 2009] Y. Bengio et al. Curriculum Learning. ICML 2009, pp. 41–48.

[Clark et al., 2022] J. Clark et al. CANINE: Pre-training an efficient tokenization-free encoder. TACL 10, 73–91.

[Conneau et al., 2020] A. Conneau et al. Unsupervised cross-lingual representation learning at scale. ACL 2020, pp. 8440–8451.

[Devlin et al., 2019] J. Devlin et al. BERT: Pre-training of deep bidirectional transformers. NAACL 2019, pp. 4171–4186.

[He et al., 2021] P. He et al. DeBERTa: Decoding-enhanced BERT with disentangled attention. ICLR 2021.

[Jiao et al., 2020] X. Jiao et al. TinyBERT: Distilling BERT for NLU. Findings of EMNLP 2020, pp. 4163–4174.

[Joshi et al., 2020] M. Joshi et al. SpanBERT: Improving pre-training by representing and predicting spans. TACL 8, 64–77.

[Lafferty et al., 2001] J. Lafferty et al. Conditional random fields. ICML 2001, pp. 282–289.

[Lample et al., 2016] G. Lample et al. Neural architectures for named entity recognition. NAACL 2016, pp. 260–270.

[Lan et al., 2020] Z. Lan et al. ALBERT: A lite BERT for self-supervised learning. ICLR 2020.

[Liu et al., 2019] Y. Liu et al. RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692.

[Loshchilov & Hutter, 2019] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. ICLR 2019.

[Ma & Hovy, 2016] X. Ma and E. Hovy. End-to-end sequence labeling via BiLSTM-CNNs-CRF. ACL 2016, pp. 1064–1074.

[Ma et al., 2020] W. Ma et al. CharBERT: Character-aware pre-trained language model. COLING 2020, pp. 39–50.

[Merity et al., 2017] S. Merity et al. Pointer sentinel mixture models. arXiv:1609.07843.

[Pruthi et al., 2019] D. Pruthi et al. Combating adversarial misspellings. ACL 2019, pp. 1601–1611.

[Sander et al., 2023] E. Sander et al. RetVec: Resilient and efficient text vectorizer. ICML 2023.

[Sanh et al., 2019] V. Sanh et al. DistilBERT, a distilled version of BERT. NeurIPS 2019 Workshop.

[Sun et al., 2020] Z. Sun et al. MobileBERT: A compact task-agnostic BERT. ACL 2020, pp. 2158–2170.

[Tjong Kim Sang & De Meulder, 2003] E. Tjong Kim Sang and F. De Meulder. CoNLL-2003 shared task. HLT-NAACL 2003, pp. 142–147.

[Vaswani et al., 2017] A. Vaswani et al. Attention is all you need. NeurIPS 2017.

[Wei & Zou, 2019] J. Wei and K. Zou. EDA: Easy data augmentation. EMNLP-IJCNLP 2019, pp. 6383–6389.

[Xiong et al., 2020] R. Xiong et al. On layer normalization in the transformer architecture. ICML 2020.

[Xue et al., 2022] L. Xue et al. ByT5: Towards a token-free future. TACL 10, 291–306.

[Zhu et al., 2020] C. Zhu et al. FreeLB: Enhanced adversarial training for NLP. ICLR 2020.

A. Appendix: β Sensitivity in Stage 3

β	Clean F1	Medium F1
0.0 (distill only)	2.5	2.9
0.5 (equal)	85.7	78.0
1.0 (class only)	84.6	74.2

Table A1: *Effect of β on CoNLL-2003 F1.*

β=0 (pure distillation) collapses to near-random F1 — the model learns to mimic teacher representations but has no signal for label assignment. β=1 (pure classification) performs well on clean text but lags at medium noise. β=0.5 achieves the best balance and is used in all main experiments.

B. Appendix: Training Convergence

Stage	Loss type	Final loss
Stage 1 (BERT → RetBERT)	Cosine distance	0.0419
Stage 2 (RetBERT → LightRet)	Cosine distance	0.1184
Stage 3 (NER fine-tuning)	Val cross-entropy	0.0504

Table A2: *Training convergence across all three stages.*

Stage 1 achieves cosine similarity of 0.958, confirming the character-level student closely matches the subword teacher. Stage 2 loss of 0.1184 reflects the inherent compression from 768-d to 256-d across 4 layers vs 12. Stage 3 val CE of 0.0504 is consistent with the 85.7 clean F1 reported.